

# Diplomado virtual PROGRAMACIÓN EN JAVASCRIPT

Guía didáctica – Módulo 5



```
der'],
];
otes);
ods'] = $quotes;
address;

pping_method']) && !
pping_method']) &&
pping_method']['code']

ta['lpa']['

or_shipping_methods');

ication/json');

382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423

    type.pause = function (e) {
    (this.paused = true)
    if (this.$element.find('.next, .prev').length &&
    this.$element.trigger($.support.transition.end
    this.cycle(true)
    }
    this.interval = clearInterval(this.interval)
    return this
    }
    Carousel.prototype.next = function () {
    if (this.sliding) return
    return this.slide('next')
    }
    Carousel.prototype.prev = function () {
    if (this.sliding) return
    return this.slide('prev')
    }
    Carousel.prototype.slide = function (type, next) {
    var $active = this.$element.find('.item.active')
    var $next = next || this.getItemForDirection
    var isCycling = this.interval
    var direction = type == 'next' ? 'left' : 'right'
    var fallback = type == 'next' ? 'first' : 'last'
    var that = this
    if (!$next.length) {
    if (!this.options.wrap) return
    $next = this.$element.find('.item')[fallback]()
    }
    if ($next.hasClass('active')) return (this.slide
    var relatedTarget = $next[0]
    var slideEvent = $.Event('slide.bs.carousel', {
    relatedTarget: relatedTarget,
    direction: direction
    })
    this.$element.trigger(slideEvent)
```

Imagen: Pixabay



Imagen: Pixabay

# Guía didáctica

## API, fetch y proyectos



## Competencia específica

Se espera que, con los temas abordados en la guía didáctica del módulo 5: API, fetchs y proyectos, el estudiante logre la siguiente competencia específica:

Aplicar el concepto de las API y fetch para la obtención de datos dinámicos y su aplicación frente a la manipulación del DOM.





# Contenidos temáticos

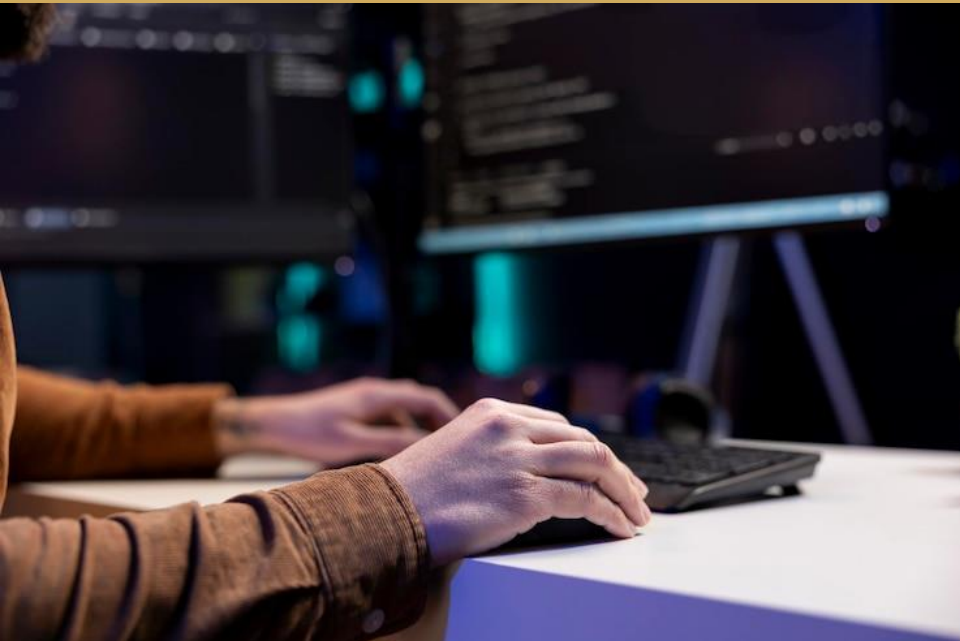


Imagen: Freepik

Los siguientes son los contenidos temáticos para desarrollar en la guía didáctica del módulo 5: API, fetch y proyectos:

- 1 API
- 2 Fetch
- 3 Proyectos



# Tema 1:

# API

# ¿Qué es una API?

«Las **API** son mecanismos que permiten a dos componentes de *software* comunicarse entre sí mediante un conjunto de definiciones y protocolos. Por ejemplo, el sistema de *software* del instituto de meteorología contiene datos meteorológicos diarios. La aplicación meteorológica de su teléfono “habla” con este sistema a través de las API y le muestra las actualizaciones meteorológicas diarias en su teléfono » (Aws Amazon, 2021).

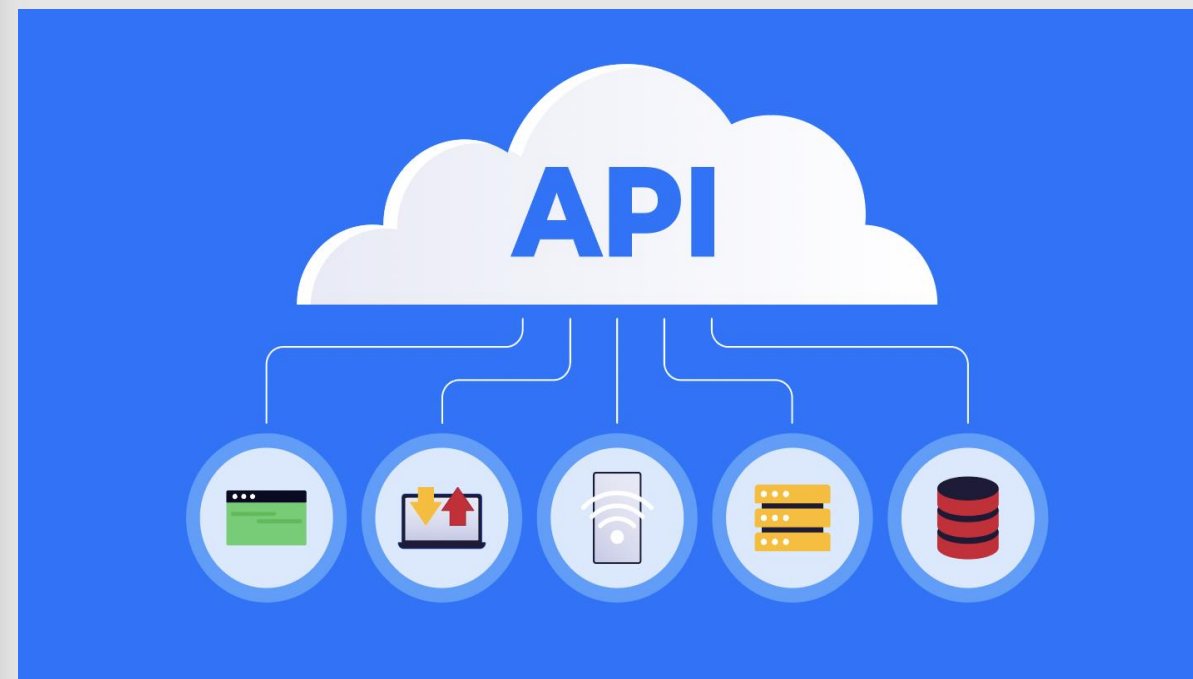


Imagen: App Master

# ¿Qué significa API?

API significa **interfaz de programación de aplicaciones**. En el contexto de las **API**, la palabra aplicación se refiere a cualquier *software* con una función distinta. La interfaz puede considerarse como un contrato de servicio entre dos aplicaciones. Este contrato define cómo se comunican entre sí mediante solicitudes y respuestas. La documentación de su **API** contiene información sobre cómo los desarrolladores deben estructurar esas solicitudes y respuestas.



Imagen: Cointic

# ¿Cómo funcionan?

La arquitectura de las API suele explicarse en términos de cliente y servidor. La aplicación que envía la solicitud se llama cliente, y la que envía la respuesta se llama servidor.

En el ejemplo del tiempo, la base de datos meteorológicos del instituto es el servidor, y la aplicación móvil es el cliente.

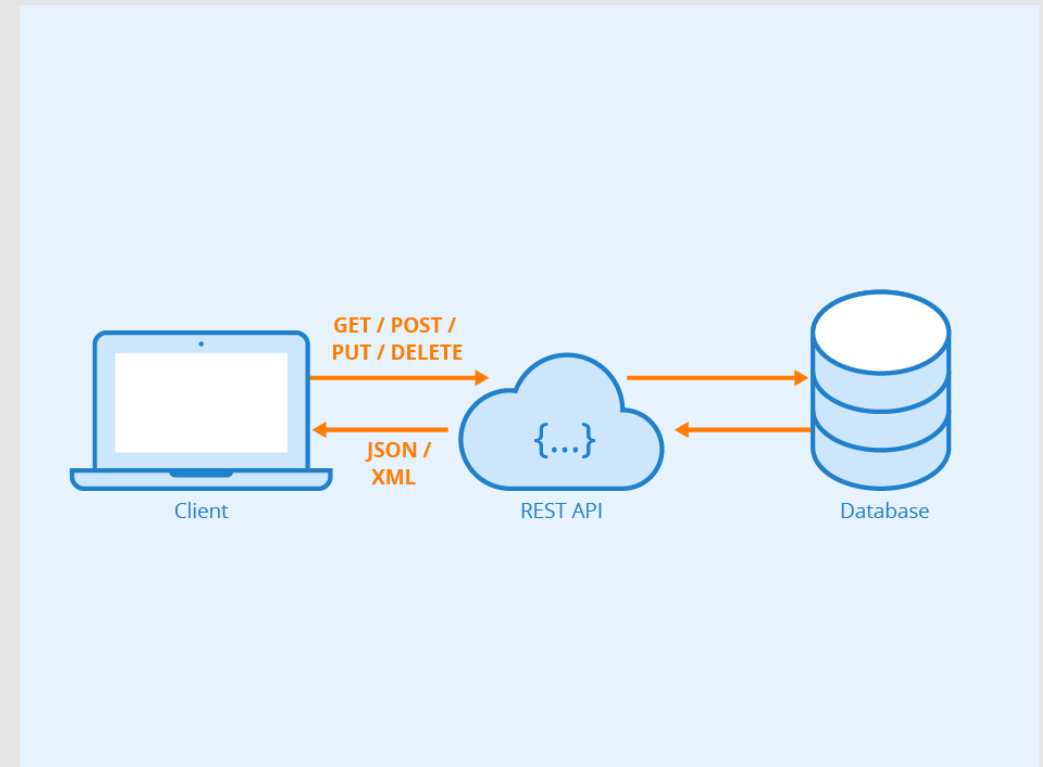


Imagen: Kinsta



# API REST

Estas son las **API** más populares y flexibles que se encuentran en la web actualmente. El cliente envía las solicitudes al servidor como datos. El servidor utiliza esta entrada del cliente para iniciar funciones internas y devuelve los datos de salida al cliente.

Veamos las **API REST** con más detalle a continuación.

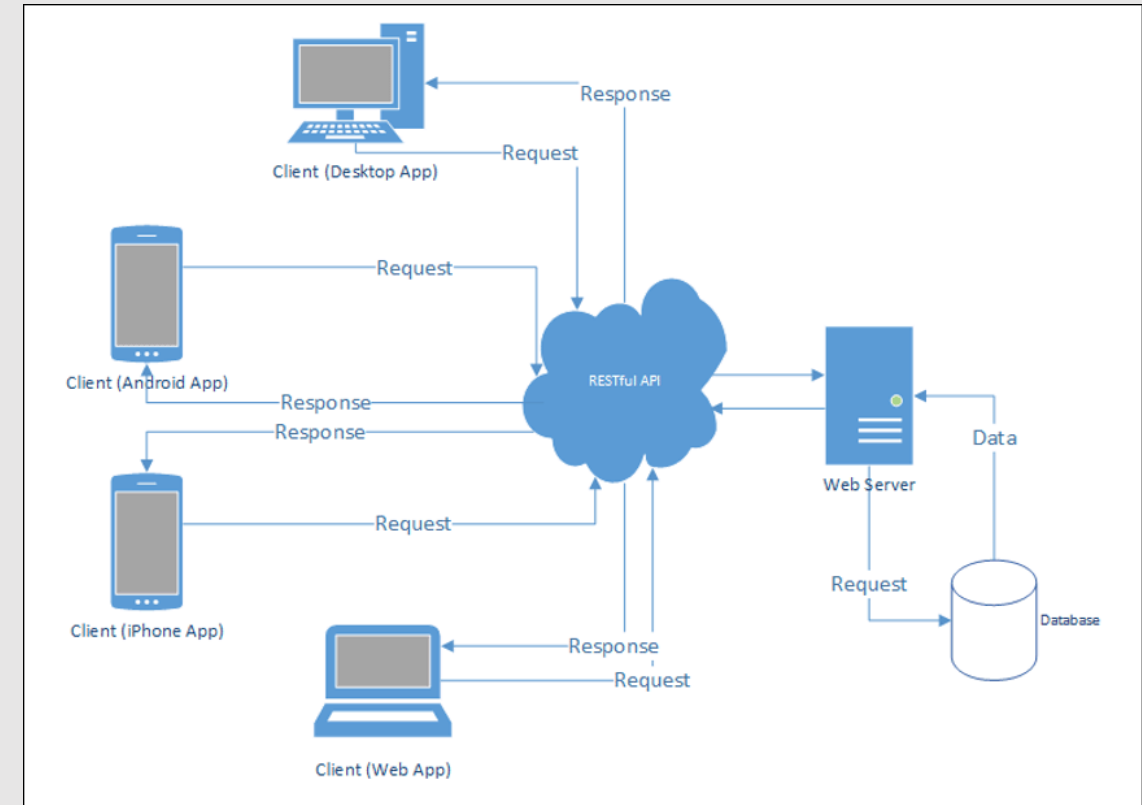


Imagen: Linuxitos

REST significa transferencia de estado representacional. REST define un conjunto de funciones como **GET**, **PUT**, **DELETE**, etc., que los clientes pueden utilizar para acceder a los datos del servidor. Los clientes y los servidores intercambian datos mediante **HTTP**.

La principal característica de la **API REST** es que no tiene estado. La ausencia de estado significa que los servidores no guardan los datos del cliente entre las solicitudes. Las solicitudes de los clientes al servidor son similares a las **URL** que se escriben en el navegador para visitar un sitio web y se conocen como *endpoints* (puntos finales). La respuesta del servidor son datos simples, sin la típica representación gráfica de una página web.

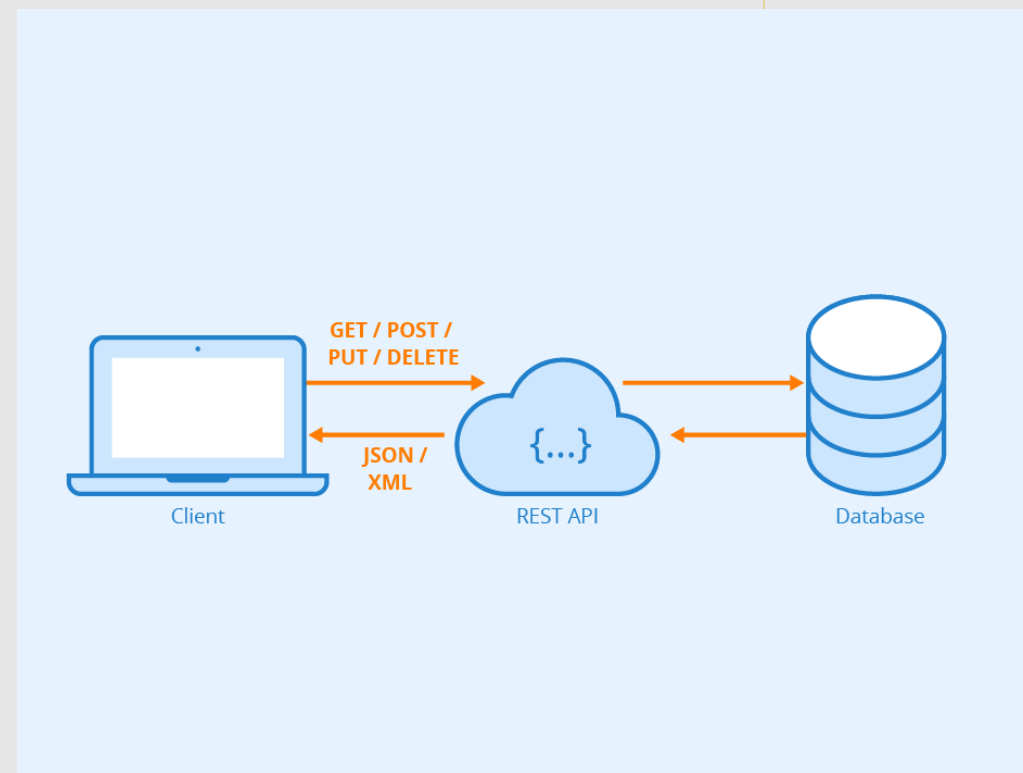


Imagen: Kinsta

# Postman

**Postman** es una herramienta de colaboración para el desarrollo de **API** que permite a los desarrolladores diseñar, probar, documentar y compartir **API** de manera más eficiente.

Con **Postman** los desarrolladores pueden enviar solicitudes **HTTP** y **HTTPS** a través de una interfaz de usuario intuitiva y ver las respuestas de estas solicitudes.

Para acceder: <https://www.postman.com/>



# Practica lo aprendido

Para practicar lo visto sobre API REST, realiza el siguiente ejercicio:

## Ejercicio

Consume todas las API en colecciones de Postman, consulta la documentación de cada una para implementar los *endpoints* más conocidos de cada una.

<https://jsonplaceholder.typicode.com/>

<https://api-colombia.com/swagger/index.html>

<https://dog.ceo/dog-api/>

# Tema 2:

# Fetch



# Fetch

**Fetch** es el nombre de una nueva API para JavaScript con la cual podemos realizar peticiones **HTTP** asíncronas utilizando promesas, de forma que el código sea un poco más sencillo y menos verboso.

La forma de hacer una petición es muy sencilla, básicamente se trata de llamar a **fetch** y pasarle por parámetro la **URL** de la petición que se va a realizar:

```
fetch('https://jsonplaceholder.typicode.com/todos/')
  .then(response => response.json())
  .then(json => console.log(json))

fetch('https://jsonplaceholder.typicode.com/todos/', {
  method: POST,
  headers: {
    "Content-Type": "application/json",
    "Authentication": "token"
  },
  body: JSON.stringify(jsonData)
})
  .then(response => response.json())
  .then(json => console.log(json))
```

# Ventajas



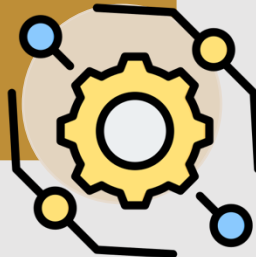
- Es más moderno y limpio que el antiguo XMLHttpRequest.

Moderno



- Soporta promesas, por lo que se puede usar con `then()` o `async/await`.

Asíncrono



- Es compatible con la mayoría de navegadores modernos.

Compatible





# Funcionamiento

1

fetch() se ejecuta y envía la solicitud.

2

Retorna una **Promesa** con un objeto Response.

3

Debemos procesar la respuesta con métodos como .json(), .text(), etc.

4

Luego podemos usar los datos.

# Tema 3:

# Proyectos

# Proyectos



Implementa fetch para el consumo de 3 API en proyectos independientes. Estos deben tener una interfaz sencilla creada con Bootstrap, además de consumir los *endpoints* más importantes de la API. Revisar los requerimientos de cada proyecto en la carpeta de “Proyectos”.

<https://pokeapi.co/>

<https://randomuser.me/>

<https://rickandmortyapi.com/>

# Referencias bibliográficas

App Master. (2022, 19 de julio). *APIs para principiantes: ¿cómo utilizar una API? Una guía completa* [imagen]. App Master. <https://tinyurl.com/2u8m57yx>

Aws Amazon. (2021, 30 de septiembre). *¿Qué es una interfaz de programación de aplicaciones (API)?* [imagen]. Aws Amazon. <https://tinyurl.com/2hv235ze>

Cointic. (2023, 12 de octubre). *¿Qué es una API?* [imagen]. Cointic. <https://tinyurl.com/2hv235ze>

Kinsta. (2025, 4 de marzo). *Cómo solucionar el «cURL Error 28: Connection Time Out».* [imagen]. Kinsta. <https://tinyurl.com/4u9tpp4k>

Linuxitos. (2022, 6 de enero). *REST API en CodeIgniter 4.* [imagen]. Linuxitos. <https://tinyurl.com/23t2nzx5>

Esta guía fue elaborada para ser utilizada con fines didácticos como material de consulta de los participantes en el diplomado virtual en PROGRAMACIÓN EN JAVASCRIPT del Politécnico de Colombia, y solo podrá ser reproducida con esos fines. Por lo tanto, se agradece a los usuarios referirla en los escritos donde se utilice la información que aquí se presenta.

## **GUÍA DIDÁCTICA 5**

M3-DV114-GU05

MÓDULO 5: API, FETCH Y PROYECTOS

© DERECHOS RESERVADOS - POLITÉCNICO DE  
COLOMBIA, 2025  
Medellín, Colombia

Proceso: Gestión Académica Virtual

Realización del texto: Diego Alejandro Palacio, docente

Revisión del texto: Comité de Revisión

Diseño: Comunicaciones

Editado por el Politécnico de Colombia

